

Integração com Cloud Computing

A integração com Cloud Computing é uma etapa essencial no desenvolvimento de aplicações modernas. O uso de serviços de nuvem permite o armazenamento seguro de dados, escalabilidade, processamento remoto e acesso global. Este texto explora como a computação em nuvem pode ser integrada a apps e discute os principais benefícios e desafios dessa abordagem.



Fonte: <https://pixabay.com/pt/vectors/icons-cloud-heart-chess-bars-873314/>

Introdução à Integração com Cloud Computing

A computação em nuvem revolucionou a maneira como os desenvolvedores criam, implementam e gerenciam aplicações. Em vez de depender de infraestrutura local, os serviços de nuvem oferecem recursos sob demanda

por meio da Internet. Esses serviços incluem armazenamento, bancos de dados, funções como serviço (FaaS), e integração com APIs de terceiros.

Benefícios da Cloud Computing

- **Escalabilidade:** Os recursos podem ser ajustados automaticamente para atender a demandas variáveis.
- **Custo-benefício:** Pagamento apenas pelo que for usado.
- **Confiabilidade:** Backup e redundância embutidos.
- **Flexibilidade:** Acesso remoto a partir de qualquer lugar com conexão à Internet.

Serviços Mais Comuns na Cloud

- **Infraestrutura como Serviço (IaaS):** Fornece máquinas virtuais, redes e armazenamento.
- **Plataforma como Serviço (PaaS):** Facilita a criação de apps com ferramentas e ambientes de desenvolvimento.
- **Software como Serviço (SaaS):** Aplicações completas acessíveis pela web.
- **Funções como Serviço (FaaS):** Permite executar códigos sem gerenciar servidores.

Desafios da Integração com a Nuvem

- **Segurança e Privacidade:** Garantir a proteção de dados armazenados.
- **Latência:** Minimizar atrasos em aplicações que dependem de conexões rápidas.
- **Custo:** Gerenciar gastos com infraestrutura e uso excessivo de recursos.

Casos de Uso de Integração com Cloud Computing

Armazenamento de Dados:

- Apps como Google Drive e Dropbox dependem da nuvem para armazenar e sincronizar arquivos.
- Backend como Serviço (BaaS):
- Ferramentas como Firebase e AWS Amplify simplificam o desenvolvimento backend para apps.

Processamento de Dados:

- Análise de grandes volumes de dados usando plataformas como Google Cloud e Azure.

Integração com APIs:

- Conectar aplicações com serviços de terceiros, como mapas ou sistemas de pagamento.

GitHub da Disciplina

Você pode acessar o repositório da disciplina no GitHub a partir do seguinte link:

<https://github.com/FaculdadeDescomplica/Pratica-Integradora-Desenvolvimento-de-Apps>. Esse espaço é o seu portal para mergulhar fundo no universo da aprendizagem interativa. Nele, você encontrará todos os códigos, além dos links para os arquivos e dados.

Conteúdo Bônus

Confira uma matéria interessante sobre como a computação em nuvem está transformando a indústria:

Título: Cloud Computing: O que é e para que serve?

Plataforma: Salesforce

Referências Bibliográficas

BIESSEK, Alessandro. Flutter for Beginners. 1. ed. Birmingham: Packt Publishing, 2020.

SEJOURNÉ, Philippe. Flutter: Aplicações Nativas Multiplataforma com Flutter e Dart. 1. ed. Rio de Janeiro: Alta Books, 2020.

ZAMMETTI, Frank. Practical Flutter: Improve your Mobile Development with Google's Latest Open-Source Framework. 1. ed. Berkeley: Apress, 2019.

Atividade Prática 13 - Integração com Cloud Computing

Título da Prática: Integração de um Aplicativo com um Serviço de Cloud Computing

Objetivos: Entender os conceitos básicos de integração de um aplicativo móvel com um serviço simples de cloud computing para armazenar e recuperar dados.

Materiais, Métodos e Ferramentas: Para realizar esta prática, utilize o Android Studio ou Flutter. Vamos simular a integração com um serviço de nuvem para armazenamento e recuperação de dados usando uma API simples.

Roteiro

1º Passo) Leia atentamente o texto.

A cloud computing (computação em nuvem) permite que os aplicativos móveis armazenem e compartilhem dados em servidores remotos, em vez de armazenar localmente no dispositivo. Muitos aplicativos utilizam a nuvem para armazenar dados como registros de usuário, configurações, ou qualquer outro tipo de dado que precisa ser acessado de diferentes dispositivos.

Neste exemplo, vamos simular a integração com um serviço simples de cloud computing usando uma **API REST**. O serviço será simulado para permitir que o aplicativo envie e recupere dados para um servidor remoto.

A API REST que usaremos para a simulação é a **JSONPlaceholder**, uma API pública para testes que oferece endpoints para criar, ler, atualizar e excluir dados (CRUD).

2º Passo) Configuração do projeto.

1. Crie um novo projeto no Android Studio ou Flutter.
2. Adicione a dependência para requisições HTTP no arquivo pubspec.yaml (Flutter) ou build.gradle (Android):

No **Flutter**:

yaml

CopiarEditar

dependencies:

http: ^0.13.3

No Android:

gradle

CopiarEditar

implementation 'com.squareup.retrofit2:retrofit:2.9.0'

implementation 'com.squareup.retrofit2:converter-gson:2.9.0'

3º Passo) Consumindo uma API REST (Simulação de Cloud Computing).

A API JSONPlaceholder oferece um endpoint para **obter posts**. Vamos consumir esse endpoint para simular a integração com a nuvem.

Aqui está um exemplo de código em **Flutter**:

dart

CopiarEditor

```
import 'package:http/http.dart' as http;
```

```
import 'dart:convert';
```

```
import 'package:flutter/material.dart';
```

```
void main() {
```

```
  runApp(MyApp());
```

```
}
```

```
class MyApp extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      home: HomeScreen(),
```

```
    );
```

```
  }
```

```
}
```

```
class HomeScreen extends StatefulWidget {
```

```
  @override
```

```
  _HomeScreenState createState() => _HomeScreenState();
```

```
}
```

```
class _HomeScreenState extends State<HomeScreen> {
```

```
  List<dynamic> _posts = [];
```

```
  @override
```

```
  void initState() {
```

```
    super.initState();
```

```
    _fetchPosts();
```

```
  }
```

```
  Future<void> _fetchPosts() async {
```

```
    final
```

```
    response
```

```
    =
```

```
    await
```

```
    http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));
```

```

if (response.statusCode == 200) {
  setState(() {
    _posts = json.decode(response.body);
  });
} else {
  throw Exception("Failed to load posts");
}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text("Integração com Cloud")),
    body: ListView.builder(
      itemCount: _posts.length,
      itemBuilder: (ctx, index) {
        return ListTile(
          title: Text(_posts[index]['title']),
          subtitle: Text(_posts[index]['body']),
        );
      },
    ),
  );
}

```

Esse código consome o serviço de API JSONPlaceholder para buscar uma lista de posts e exibi-los na interface do usuário.

4º Passo) Responda às questões abaixo:

A) O que é o JSONPlaceholder?

- B)** Qual função a biblioteca `http` desempenha no código Flutter?
- C)** Como um aplicativo consome dados da nuvem por meio de uma API REST?
- D)** O que a função `setState()` faz no código?

Gabarito Atividade Prática

- A)** O **JSONPlaceholder** é um serviço de API pública para testes, que fornece **dados simulados** para desenvolvedores testarem requisições HTTP sem precisar de um backend real.
- B)** A biblioteca **`http`** no Flutter é responsável por **fazer requisições HTTP para APIs externas**, permitindo que o aplicativo envie e receba dados de servidores remotos.
- C)** Um aplicativo consome dados da nuvem por meio de uma API REST **usando a função `fetchPosts()`** ou funções similares que fazem requisições HTTP para obter informações do servidor.
- D)** A função `setState()` no código Flutter **atualiza a interface gráfica com novos dados**, garantindo que a UI reflita as mudanças no estado da aplicação.

Comentários e Conclusões

1. O que aprendeu com esta atividade?
2. Como a integração com a nuvem pode melhorar a funcionalidade de um aplicativo móvel?
3. Quais dificuldades você encontrou ao configurar a integração com uma API REST?